

```

self.labels = labels
self.listIDs = listIDs

def __len__(self):
    # To get the total number of
    samples
    return len(self.listIDs)
    def __getitem__(self, index):
        # get one sample of database
        # get the required sample
        newID = self.listIDs[index]
        # Load the data and get the
        corresponding label
        X = torch.load('data/' + newID +
        '.pt')
        y = self.labels[newID]

        # return the sample with label
        return X, y

```

Now the DataLoader class of PyTorch provides an interface to use the data set generated above from the Dataset class.

```

# Check for GPU and enable GPU or CPU
device accordingly
use_gpu = torch.cuda.is_available()
device = torch.device("cuda:0" if use_gpu
else "cpu"
torch.backends.cudnn.benchmark = True

```

```

loaderParams = {'batch_size': 64,
'shuffle': True, 'num_workers': 6}

```

The above parameters are passed to the DataLoader class and their purpose is given below.

batch_size: This denotes the number of samples contained in each generated batch; generally, this is multiples of 8 (8, 32, 64, 128...).

shuffle: The samples in the batch are shuffled for each epoch, so that batches between epochs are not similar for training the model. This allows the model to be more robust during training. Generally, it is set to *True*. Setting it to *False* will allow the use of the same samples in batches across training epochs.

num_workers: This denotes the number of processes that generate batches in parallel; a high number of

workers will allow CPU computations to be managed efficiently.

```

maxEpochs = 50
# Define or get the data samples with
training samples and validation samples
# separately and store them in
datapartition dictionary and simlary
their
# corresponding labels in datalabels
dictionary.
datapartition = {'train': ['id1',
'id2', 'id3', 'id4', 'id5'],
'validation': ['id6', 'id7', 'id8',
'id9']}
datalabels = {'id1': 0, 'id1': 2,
'id3': 0, 'id4': 3, 'id5': 1, 'id6': 0,
'id7': 2, 'id8': 0, 'id9': 3 }

# get the data generation object
trainingSet = newDataset(datapartition[
'train'], datalabels)
trainingGenerator = torch.utils.data.
DataLoader(trainingSet, **loaderParams)

validationSet = Dataset(partition['vali
dation'], datalabels)
validationGenerator = torch.utils.
data.DataLoader(validationSet,
**loaderParams)

# To get each batch of samples from the
training dataset
for i, batch in
enumerate(trainingGenerator):
    print (i, batch)

# To get each batch of samples from the
validation dataset
for i, batch in
enumerate(validationGenerator):
    print (i, batch)

```

```

# To get each batch of samples and
their labels from the training dataset
for batch, labels in trainingGenerator:
    print (batch, labels)

```

```

# To get each batch of samples and
their labels from the validation
dataset
for batch, labels in
validationGenerator:
    print (batch, labels)

```

Data is important for machine learning and deep learning algorithms to perform better. Quality data is what is needed to get a good performing model. Acquiring data, cleaning the data for quality and annotating it is a tedious task, and consumes a considerable amount of time if started from scratch. Hence, pre-built data sets save time and allow one to think about improving the learning methods rather than concentrating on acquiring data. Most of the machine learning and deep learning frameworks or platforms, therefore, provide built-in data sets as part of the package. These data sets generally come in all flavours to be used for various applications like classification, object recognition, detection, segmentation, caption generation, sentiment analysis, emotion detection and action detection. Moving a step ahead, a few frameworks like PyTorch have improved the way the data set is handled for building models. This has significantly helped data scientists to save time, build high performing models, develop insights, and make considerable progress in deep learning and machine learning technologies. 

By: B.N. Chandrashekar and Srinivas Chakravarthy Yeeda

B.N. Chandrashekar is a principal consultant and researcher with hands-on programming experience. He has around two decades of industry experience. With a master's degree from IISc, he specialises in embedded, retail, e-commerce and AI driven technology.

Srinivas Chakravarthy is technical product manager with ABB Ability Innovation Center. He works in the area of distributed control systems and industrial automation. He has interests in machine learning and deep learning. He is an avid programmer with hands-on experience in ML frameworks.